TERM PROJECT PROBLEM STATEMENT

DQN-Based Traffic Signal Control with Distribution-Shift Detection and Mitigation

Team Members

Name	Roll Number
Vivek Reddy Pallaka	AM.SC.U4AIE23050
Ayyappa Reddy	AM.SC.U4AIE23046
Sriram	AM.SC.U4AIE23014

Abstract

Signalized intersections are a major bottleneck in urban traffic. Fixed-time signals run on a preset cycle regardless of how much traffic is actually present, and max-pressure controllers, while more responsive, still follow a hand-coded rule rather than learning from experience. Both approaches struggle when traffic patterns don't match what they were tuned for.

This project builds a Deep Q-Network (DQN) agent that controls a single intersection in SUMO, learning directly from simulated traffic rather than following fixed rules. But getting a working DQN agent is only half the problem we're looking at. Once such an agent is trained, it usually assumes the world will keep looking the way it did during training — and in practice it won't. A sensor can start dropping readings, a signal actuator can lag behind commands, or the underlying traffic dynamics can simply change. Most RL traffic controllers have no way of noticing this has happened, so we are also building a layer on top of the DQN agent that can flag when conditions have shifted, figure out roughly what kind of shift it is, and apply a fix suited to that specific problem.

We compare the DQN agent against FixedTime and MaxPressure baselines over several random seeds, using average waiting time, delay and queue length as the main metrics. The shift-detection layer is checked separately on detection speed, false alarms, how often it correctly identifies the type of shift, and how much performance is recovered once a mitigation kicks in. Every claim of "this part works" in this project is meant to be backed by an actual file on disk — a CSV, a log, a saved model — not just a description of what was supposed to happen.

1. Introduction and Problem Background

Congestion at signalized intersections costs commuters time, wastes fuel and adds to emissions. The traffic engineering answer for decades has been fixed-time cycles or, somewhat better, max-pressure control, where the phase is chosen based on which approach has the most built-up pressure right now. These are easy to reason about and cheap to deploy, but neither one improves with experience or adapts when traffic patterns change from what the engineer assumed.

Reinforcement learning treats the problem differently. Instead of a rule, an agent watches the intersection's state — queue lengths, vehicle positions, per-lane waiting times — and picks a phase, getting rewarded when congestion goes down. Given enough interaction with a simulator, the agent can end up outperforming a fixed-rule controller, at least under the conditions it trained on.

That last part is the catch. A controller trained under one set of conditions has no guarantee of working once those conditions change — a sensor drops out, an actuator responds late, speed limits get revised, whatever it is. There's

usually no alarm bell; the controller just quietly gets worse. This project looks at both halves of that problem: building a DQN controller that's actually competitive, and building a way to catch and respond to this kind of shift after deployment.

2. Problem Statement

Most RL-based traffic signal controllers in the literature are trained and tested in one fixed simulation setting, and the reported numbers implicitly assume deployment will look the same. Real intersections don't cooperate — sensors fail, actuators lag, traffic dynamics drift — and none of that is something the agent has ever seen.

So there are really two problems here. First, get a DQN agent that reliably beats FixedTime and MaxPressure under normal conditions. Second — and this is the part that's easy to skip — build something that notices when the operating conditions have moved away from training conditions, works out roughly what changed, and does something targeted about it rather than just continuing to run the original policy blind. Concretely, that means:

1. Reproducing a DQN traffic-signal baseline and checking it against FixedTime and MaxPressure across multiple seeds.
2. Injecting realistic shifts at the observation, transition, action, and reward levels of the environment.
3. Catching the onset of a shift mid-episode using a statistical monitor.
4. Working out which of the four shift types occurred, using a trained classifier.
5. Applying a recovery strategy matched to the shift type and measuring how much performance comes back.

None of this is trivial. Traffic is noisy to begin with, so a real shift can look a lot like ordinary variation in the first few seconds — early detection is genuinely ambiguous. And a fix that helps doesn't always help everywhere: stretching green time for one direction to compensate for a shift can starve the cross-traffic instead.

3. Objectives

Broken down, what we're trying to get done is:

- Train and evaluate a DQN agent for single-intersection control in SUMO, across multiple seeds.
- Benchmark it against FixedTime and MaxPressure using standard traffic metrics.
- Build a shift-injection engine covering observation, transition, action, and reward-level shifts.
- Build a monitor that catches the onset of a shift without too many false alarms and without too much delay.
- Build a classifier that can tell which of the four shift types is occurring.
- Design a mitigation strategy per shift type and measure how much of the lost performance it recovers.
- Back every completion claim with something actually on disk, not a description of intended behaviour.

4. Why Reinforcement Learning?

Traffic signal control is sequential by nature — whatever phase you pick now affects the queue at the next step, and the one after that. A good controller has to think a few steps ahead, not just react to the current snapshot. That's

basically what RL is for: an agent that improves its policy through repeated interaction with an environment and a reward signal, rather than following a rule someone wrote down in advance.

We use DQN specifically because the state here — queue lengths, waiting times, vehicle positions across lanes — is continuous and reasonably high-dimensional, while the actions (which of a handful of signal phases to run) are discrete. That combination is exactly what DQN's function approximation over Q-values is built to handle.

4.1 Why Bother with Distribution-Shift Awareness?

Almost all RL traffic-control papers report a number under the same conditions the agent trained on and stop there. We think that's not enough on its own — a controller that only works when nothing has changed is of limited use once it's actually deployed. Rather than treat robustness as an afterthought, this project builds detection, diagnosis and mitigation in from the start, so the system can fail visibly and recover instead of failing silently.

5. Literature Survey

S.No	Research Paper	Methodology / Approach	Key Contribution	Research Gap	Link
1	Using a Deep Reinforcement Learning Agent for Traffic Signal Control (Genders & Razavi, 2016)	Deep Q-learning with a discrete traffic state encoding (DTSE) as input to a CNN, trained with experience replay.	Established a deep RL adaptive signal controller that substantially reduces delay and queue length versus simpler learning and fixed baselines.	Trained and evaluated only under stationary simulation conditions; does not consider deployment-time distribution shift.	arxiv.org/abs/1611.011
2	Towards Robust Deep RL for Traffic Signal Control: Demand Surges, Incidents and Sensor Failures (Rodrigues & Azevedo, 2019)	Open-source callback-based framework evaluating deep RL controllers under injected demand surges, incidents, and sensor failures.	Identifies specific failure modes of RL-based signal control under real-world disruptions and proposes mitigation ideas per disruption type.	Disruptions are handled ad hoc per scenario; no unified pipeline to detect, diagnose, and then select a mitigation strategy online.	ar5iv.arxiv.org/html/190
3	Reinforcement Learning Benchmarks for Traffic Signal Control — RESCO (Ault & Sharon, 2021)	Standardized SUMO/TraCI-based benchmark suite comparing RL controllers (IDQN,	Provides a reproducible evaluation protocol and baseline implementations widely adopted for benchmarking RL-based traffic signal control.	Assumes stationary, nominal evaluation conditions throughout; does not address robustness to distribution shift.	github.com/Pi-Star-Lab

		MPLight, IPPO, etc.) against FixedTime and Max-Pressure baselines.			
4	RobustLight: Improving Robustness via Diffusion RL for Traffic Signal Control (Li et al., ICML 2025)	Plug-and-play diffusion model that restores corrupted or missing sensor readings online before they reach the RL controller.	Improves controller robustness to noise, missing data, and adversarial attacks without modifying the underlying TSC platform.	Addresses observation-level corruption only, via restoration; does not diagnose the type of disruption or generalize to action/transition/reward-level shifts.	proceedings.mlr.press/
5	Sim2Signal: Sim-to-Real Benchmarks for Traffic Signal Control (2026)	Benchmark decomposing the sim-to-real gap into observation, action, transition, and reward gaps, inducing each in isolation under a shared protocol.	Provides a taxonomy of shift types (matching this project's observation/transition/action/reward structure) and evaluates 18 mitigation methods across calibrated real-world networks.	Focuses on benchmarking existing mitigation methods offline rather than building an integrated online detect-diagnose-mitigate agent for a live controller.	arxiv.org/abs/2609.016
6	Robustness of RL-Based Traffic Signal Control under Incidents: A Comparative Study / T-REX (2025)	SUMO-based simulation framework modeling incident-driven congestion propagation via rerouting, speed adaptation, and lane-changing, used to compare RL-TSC methods.	Extends robustness evaluation to network-level incident scenarios and proposes robustness-specific evaluation metrics beyond standard efficiency measures.	Centered on incident scenarios specifically, rather than a general four-way (observation/transition/action/reward) shift taxonomy with automated online detection and diagnosis.	arxiv.org/pdf/2506.138

6. Datasets and Simulation Environment

One thing worth clarifying up front: this isn't a supervised-learning project, so there's no labeled dataset in the usual sense. Traffic — vehicle arrivals, routes, demand — is generated procedurally inside SUMO via TraCI, using the same single-intersection network and demand setup as Vidali's open-source DQN traffic-signal baseline. Each seed gives a different randomly generated demand pattern, which is what lets us compare the DQN agent, FixedTime, and MaxPressure fairly across seeds.

The shift-injection engine doesn't need a separate dataset either. It perturbs the same SUMO environment directly — at the observation, transition, action, or reward level — so nominal and shifted runs can be compared inside one controlled simulation instead of us having to go find or fabricate real shifted traffic data.

For context, some of the broader literature does test on networks calibrated to real cities — RESCO uses Ingolstadt (InTAS) and Cologne (TAPAS) networks, and Sim2Signal uses ten networks calibrated from five real locations. We're noting these as possible directions if this project extends to multiple intersections or real-world calibration later, not as something the current single-intersection setup needs right now.

7. Gap Analysis

RL-based traffic control already performs well under stable, nominal conditions, and a handful of separate papers have started poking at robustness — sensor failure here, sim-to-real gaps there. What's missing is any of this being tied together. Specifically:

1. Detection and mitigation are rarely combined — papers either flag that something's off, or apply a fix, but not both in sequence.
2. Most robustness work looks at one failure mode at a time (sensor failure, or incidents) instead of a shared taxonomy across observation, transition, action, and reward shifts.
3. Mitigation is usually applied the same way regardless of what actually went wrong, rather than being chosen based on a diagnosis of which part of the MDP shifted.
4. Recovery from a shift tends to get described in words rather than measured as a number against an unmitigated baseline.
5. Almost nobody builds an explicit fallback for the case where the RL policy just collapses under a shift too severe to recover from normally.

The four-module pipeline in this project — injection, monitoring, diagnosis, mitigation — is built to close these gaps, with every stage checked against something concrete on disk rather than taken on faith.

8. Proposed Research Direction

The work is split into three phases. Phase 1 gets the DQN baseline working and validated against the classical controllers. Phase 2 builds out the four shift-related modules. Phase 3 — integration, broader evaluation, final write-up — will be detailed in a later review once Phase 2 is settled.

1. Baseline Reproduction (Phase 1): train the DQN agent across multiple SUMO seeds, evaluate it with a frozen greedy policy, and compare against FixedTime and MaxPressure on waiting time, delay, and queue length.

2. Shift-Injection Engine (Module 1): build configurable shift injectors — sensor bit-flip/dropout for observation, speed/deceleration scaling for transition, actuator latency/stuck actions for action, switching/queue penalties for reward.
3. Shift Monitor (Module 2): track rolling features — occupancy, approach velocity, TD error, Q-spread, queue length — and score them against a calibrated nominal baseline to catch a shift starting.
4. Diagnosis Classifier (Module 3): once a shift is flagged, classify which of the four types it is, using features drawn from the monitored episode.
5. Mitigation Module (Module 4): apply a strategy matched to the diagnosed shift type — a consensus filter for observation shifts, a velocity-adaptive phase stretcher for transition shifts, a predictive lookahead buffer for action-latency shifts, and a fallback governor that hands off to MaxPressure if the policy collapses outright — then measure how much performance is recovered.

9. Expected Evaluation Metrics

- Average vehicle waiting time
- Average vehicle delay
- Average queue length
- Cumulative (negative) reward
- Shift-detection latency (steps to detection)
- Nominal false-positive detection rate
- Shift-diagnosis classification accuracy
- Post-mitigation recovery ratio (relative to unmitigated performance under shift)

The DQN agent gets compared against FixedTime and MaxPressure under normal conditions, and separately, the full detect-diagnose-mitigate pipeline gets evaluated under each of the four injected shift types against an unmitigated, shifted run.